

Question:

In the Insertion Sort algorithm, analyze the time complexity in the best, average, and worst-case scenarios. Additionally, modify the algorithm to work in reverse order, such that it sorts the array in descending order instead of ascending. Provide the modified pseudocode and Python code for the reverse sorting implementation.

Pseudo Code

INSERTION-SORT(A)

1. For $j = 2$ to $A.length$
2. $key = A[j]$
3. // Insert $A[j]$ into the sorted sequence $A[1..j-1]$
4. $i = j - 1$
5. While $i > 0$ and $A[i] > key$
6. $A[i+1] = A[i]$
7. $i = i - 1$
8. $A[i+1] = key$

Answer def insertion_sort(A):

 # Iterate through the array starting from the second element

 for j in range(1, len(A)):

 key = A[j]

 i = j - 1

 # Move elements of $A[0..j-1]$ that are greater than key to one position ahead

 while i >= 0 and $A[i] > key$:

$A[i + 1] = A[i]$

```
i -= 1
```

```
# Place key at the correct position
```

```
A[i + 1] = key
```

```
return A
```

```
# Test case
```

```
array = [12, 11, 13, 5, 6]
```

```
print("Sorted array in ascending order:", insertion_sort(array))
```